

Trabalho 2 — Compactação e Descompactação de Texto (Huffman)

INF 1010 — Estruturas de Dados Avançadas (PUC-Rio)

1. Identificação

- **Aluno:** Daniel Santana Souza — **Matrícula:** 2310995 — Turma 3WB
- **Disciplina:** INF 1010 — Estruturas de Dados Avançadas
- **Trabalho 2:** Compactador / Descompactador de arquivos texto usando o algoritmo de Huffman

2. Objetivo

O trabalho consiste em escrever, em C, dois programas capazes de **compactar e descompactar arquivos de texto sem perda** usando o algoritmo de Huffman. A ideia central do algoritmo é varrer o arquivo original, levantar a frequência com que cada caractere ocorre e substituir a representação fixa de 8 bits por código por uma codificação de **tamanho variável** em que os caracteres mais frequentes recebem códigos mais curtos e os menos frequentes recebem códigos mais longos. A árvore binária de prefixos (Trie de Huffman) garante que nenhum código é prefixo de outro, o que dispensa qualquer separador entre dois códigos consecutivos no fluxo de bits.

Os programas devem:

- ler o arquivo de entrada e construir o histograma de frequências dos bytes;
- construir a árvore de Huffman a partir desse histograma;
- gerar a tabela de códigos (caractere → código binário, tamanho);
- gravar o arquivo compactado de modo que o descompactador consiga recuperar o original exatamente, sem perda;
- exibir, no final do processo, a tabela com caractere, frequência e código binário usada pelo compactador.

3. Estrutura do programa

O trabalho está organizado em quatro arquivos fontes e dois cabeçalhos. Os módulos isolam três responsabilidades: a árvore de Huffman, a I/O em bits e os programas principais (compactador e descompactador).

huffman.h / huffman.c

Módulo que concentra tudo o que se refere à árvore de Huffman:

- **Tipos públicos.** `t_hnode` representa um nó da árvore (com frequência, símbolo e dois filhos); `t_codigo` guarda um código em binário como string de '0'/'1' e seu tamanho em bits.
- `huffman_constroi(freq)` — recebe um vetor de 256 posições com a frequência de cada byte e devolve a raiz da árvore de Huffman. Internamente usa uma fila ordenada (lista ligada, inserção ordenada) para sempre retirar as duas subárvores de menor frequência e combiná-las em uma nova raiz. Trata o caso degenerado de um único símbolo distinto: nesse caso a árvore tem dois nós (uma folha real e uma folha "fantasma" de frequência 0), o que garante um código de pelo menos 1 bit para o único símbolo.
- `huffman_gera_tabela(arv, tab)` — preenche o vetor de 256 entradas `t_codigo` percorrendo a árvore: a descida pela esquerda acrescenta o bit 0 e pela direita o bit 1. Ao chegar em uma folha o código acumulado é copiado para a entrada correspondente ao símbolo.
- `huffman_imprime_tabela(freq, tab, out)` — imprime a tabela pedida no enunciado (caractere, frequência, número de bits, código binário). Apenas as linhas com frequência > 0 são exibidas.
- `huffman_escreve_cabecalho / huffman_le_cabecalho` — gravam e recuperam o cabeçalho do arquivo compactado descrito na Seção 4.2.
- `huffman_libera(arv)` — libera recursivamente a memória.

`bitio.h / bitio.c`

Módulo de leitura e escrita de **bits** em arquivo. Como o sistema de arquivos lê e escreve bytes, este módulo é responsável por empacotar o fluxo de bits em bytes na escrita e desempacotá-los na leitura. As estruturas `t_bitwriter` e `t_bitreader` mantêm o byte atual e o número de bits já consumidos/escritos. A convenção é gravar o primeiro bit no **bit mais significativo** do primeiro byte, o segundo bit no bit 6 e assim por diante. Quando o byte fica cheio, é descarregado em `stdio`. Funções públicas: `bw_init`, `bw_write_bit`, `bw_write_bits`, `bw_flush`, `br_init` e `br_read_bit`.

`compactador.c`

Programa principal do compactador. Conta as frequências dos bytes do arquivo de entrada, constrói a árvore, gera a tabela, escreve o cabeçalho no arquivo de saída e, em uma segunda passagem pelo arquivo de entrada, escreve os códigos de Huffman bit a bit usando o módulo `bitio`. No final imprime na tela a tabela de frequências/códigos e um resumo com bits originais, bits codificados, taxa de compressão e tamanho final do arquivo gerado.

descompactador.c

Programa principal do descompactador. Lê o cabeçalho do arquivo compactado, **reconstrói a mesma árvore de Huffman** usada na compactação (a partir das mesmas frequências, com a mesma fila ordenada) e percorre o fluxo de bits restante: cada 0 desce pela esquerda, cada 1 desce pela direita; ao chegar em uma folha, o símbolo é gravado no arquivo de saída e o cursor volta para a raiz da árvore. O laço termina quando o número de símbolos gravados iguala o total registrado no cabeçalho — assim os bits de padding do último byte não são interpretados como símbolos.

4. Solução

4.1. Algoritmo de Huffman

A construção da árvore segue o procedimento clássico:

1. Conte a frequência de cada byte do arquivo (`freq[256]`).
2. Para cada byte com frequência > 0 , crie uma folha contendo o byte e a frequência. Insira todas as folhas em uma fila ordenada por frequência crescente.
3. Enquanto houver mais de uma árvore na fila:
 - retire as duas árvores de menor frequência, a e b ;
 - crie um nó interno cuja frequência é $a \rightarrow \text{freq} + b \rightarrow \text{freq}$ e cujos filhos esquerdo e direito são a e b ;
 - insira esse nó interno de volta na fila ordenada.
4. A árvore que sobra é a árvore de Huffman.

Os códigos são lidos descendo da raiz às folhas: cada descida pela esquerda acrescenta o bit 0 e cada descida pela direita acrescenta o bit 1. Como as folhas só existem nas pontas, **nenhum código é prefixo de outro** — **basta percorrer a árvore para decodificar o fluxo** de bits sem ambiguidade.

A construção da árvore tem complexidade $O(n^2)$ na implementação adotada (lista ligada ordenada), onde $n \leq 256$ é o número de símbolos distintos. Como o alfabeto é fixo e pequeno, essa escolha foi preferida por simplicidade — uma heap binária reduziria a complexidade para $O(n \log n)$, porém com pouco ganho prático neste problema.

4.2. Formato do arquivo compactado

O cabeçalho do arquivo `.huf` contém todas as informações necessárias para que o descompactador reconstrua a árvore — não é preciso gravar a árvore propriamente dita, basta gravar as frequências e usar a **mesma lógica de construção** dos dois lados. Como a fila ordenada é

estável (em caso de empate de frequência, a ordem de inserção é preservada e a inserção segue a ordem crescente dos bytes), o compactador e o descompactador chegam exatamente à mesma árvore.

Bytes do arquivo, em little-endian:

offset	conteudo	descricao
0..3	"HUFF"	assinatura (4 bytes ASCII)
4..7	total (uint32)	nro. de caracteres do original
8..9	n (uint16; 0 == 256)	nro. de simbolos distintos
10..	n entradas de 9 bytes:	
	1 byte: simbolo	byte do alfabeto
	8 bytes: freq (u64)	frequencia do simbolo
...	fluxo de bits	codigos de Huffman concatenados

Guardar o campo total (número de símbolos do original) é essencial porque o fluxo de bits termina com até 7 bits de padding no último byte (preenchidos com zeros). Sem o total, esses zeros poderiam ser interpretados como mais um símbolo válido. Com o total, o descompactador para de ler assim que recupera o número certo de símbolos.

4.3. Saída do programa para a frase de teste

Foi gerado o arquivo frase.txt contendo exatamente a frase pedida no enunciado:

```
AS ESTRUTURAS DE DADOS SAO FUNDAMENTAIS PARA A ORGANIZACAO E A MANIPULACAO EFICIENTE DAS
INFORMACOES
```

A execução do compactador produz a tabela de frequências e códigos abaixo (exibida na tela conforme pedido pelo enunciado):

```
=== Compactador de Huffman ===
Arquivo de entrada : frase.txt
Arquivo de saída   : frase.huf
Total de caracteres no arquivo original: 100
```

Tabela de frequencias e codigos:

Caractere	Frequencia	Bits	Codigo
' ' (espaco)	14	3	101
'A'	18	2	00
'C'	4	5	11100
'D'	5	5	11111
'E'	8	4	1100
'F'	3	5	01011
'G'	1	6	010100
'I'	6	4	0110
'L'	1	6	010101
'M'	3	5	10000

'N'	6	4	0111
'O'	7	4	1001
'P'	2	6	100011
'R'	5	4	0100
'S'	8	4	1101
'T'	4	5	11101
'U'	4	5	11110
'Z'	1	6	100010

Resumo:

Bits originais (8 por caractere): 800 bits (100 bytes)
 Bits codificados (sem cabeçalho): 383 bits (47 bytes uteis,
 arredondado para 48 byte(s))
 Taxa de compressão (so dados) : 52.12%
 Tamanho final do arquivo .huf : 220 bytes (com cabeçalho)

Observa-se que os caracteres mais frequentes ('A' com 18 ocorrências e o espaço com 14) recebem os códigos mais curtos (2 e 3 bits, respectivamente), enquanto os menos frequentes ('G', 'L', 'Z', cada um com 1 ocorrência) recebem os códigos mais longos (6 bits). O conjunto de códigos satisfaz a propriedade de prefixo: nenhum dos códigos da tabela é prefixo de outro.

Os 100 caracteres do texto original ocupariam **800 bits** com o ASCII estendido de 8 bits. Os mesmos 100 caracteres em Huffman ocupam apenas **383 bits**, uma redução de **52,12% no fluxo de dados**.

4.4. Saída do descompactador

A execução do descompactador sobre o arquivo frase.huf produzido acima reconstrói exatamente o mesmo texto:

=== Descompactador de Huffman ===

Arquivo compactado : frase.huf (220 bytes)

Arquivo de saída : frase.out

Total de caracteres a recuperar: 100

Tabela de frequências e códigos (recuperada do cabeçalho):

Caractere	Frequencia	Bits	Codigo

' ' (espaco)	14	3	101
'A'	18	2	00
'C'	4	5	11100
'D'	5	5	11111
'E'	8	4	1100
'F'	3	5	01011
'G'	1	6	010100
'I'	6	4	0110
'L'	1	6	010101
'M'	3	5	10000
'N'	6	4	0111
'O'	7	4	1001
'P'	2	6	100011
'R'	5	4	0100

'S'	8	4	1101
'T'	4	5	11101
'U'	4	5	11110
'Z'	1	6	100010

Arquivo decodificado com sucesso (100 caracteres).

A tabela exibida pelo descompactador é **idêntica** à do compactador, o que confirma que a árvore foi reconstruída corretamente a partir do cabeçalho. Em seguida, `diff frase.txt frase.out` confirma que o arquivo decodificado é byte a byte igual ao original:

```
$ diff -s frase.txt frase.out
Files frase.txt and frase.out are identical
```

4.5. Teste com o arquivo do EAD

O segundo teste usa o arquivo de texto disponibilizado no EAD (texto.txt, com 4740 bytes contendo letras de músicas em maiúsculas, espaços, vírgulas implícitas removidas, CRLF entre linhas).

```
=== Compactador de Huffman ===
Arquivo de entrada : texto.txt
Arquivo de saída   : texto.huf
Total de caracteres no arquivo original: 4740
```

Tabela de frequências e códigos:

Caractere	Frequencia	Bits	Codigo

'\n' (nova linha)	188	5	11010
'\r' (CR)	188	5	11011
' ' (espaco)	799	3	111
'A'	306	4	1000
'B'	41	6	001000
'C'	84	6	101011
'D'	144	5	01011
'E'	383	3	000
'F'	78	6	101010
'G'	52	6	001001
'H'	175	5	10111
'I'	227	4	0011
'J'	1	9	010101000
'K'	124	5	01001
'L'	161	5	10100
'M'	64	6	010100
'N'	302	4	0111
'O'	370	4	1100
'P'	31	8	01010101
'R'	171	5	10110
'S'	160	5	10011
'T'	283	4	0110
'U'	117	5	01000
'V'	39	7	0101011
'W'	105	5	00101

'Y'	146	5	10010
'Z'	1	9	010101001

Resumo:

```

Bits originais (8 por caractere): 37920 bits (4740 bytes)
Bits codificados (sem cabeçalho): 20346 bits (2543 bytes uteis,
                                arredondado para 2544 byte(s))
Taxa de compressao (so dados)   : 46.34%
Tamanho final do arquivo .huf   : 2797 bytes (com cabeçalho)

```

Em seguida, descompactando texto.huf o arquivo recuperado é idêntico ao original:

```

$ ./descompactador texto.huf texto.out
...
Arquivo decodificado com sucesso (4740 caracteres).

$ diff -s texto.txt texto.out
Files texto.txt and texto.out are identical

```

O ganho é menor que no caso da frase isolada (46,34% contra 52,12%) porque o texto inclui caracteres `\r` e `\n` (CRLF) e uma variedade maior de letras, o que diminui a concentração das frequências e encurta menos os códigos médios. Mesmo assim, o arquivo final (2797 bytes incluindo cabeçalho) é cerca de **41% menor** que o original (4740 bytes).

4.6. Comparação com a codificação de 5 bits descrita no enunciado

O enunciado começa descrevendo uma codificação simples de 5 bits por caractere (ganho fixo de 3 bits/caractere em relação ao ASCII). Aplicada à frase de teste, esse esquema produziria **500 bits** (100×5). A codificação de Huffman produziu **383 bits** para os mesmos 100 caracteres, ou seja, em média **3,83 bits/caractere** — uma redução adicional de **23%** sobre o esquema simples e de **52%** sobre o ASCII de 8 bits. O ganho vem justamente de explorar a distribuição real das frequências (A aparece 18 vezes, Z só uma).

5. Observações e conclusões

Como compilar

A partir do diretório trabalho2_huffman:

```

gcc -Wall -Wextra -O2 huffman.c bitio.c compactador.c -o compactador
gcc -Wall -Wextra -O2 huffman.c bitio.c descompactador.c -o descompactador

```

Os comandos acima compilam sem nenhum warning.

Como executar

```
./compactador <arquivo_original> <arquivo_compactado.huf>
./descompactador <arquivo_compactado.huf> <arquivo_recuperado>
```

Exemplos efetivamente executados durante o desenvolvimento:

```
./compactador frase.txt frase.huf
./descompactador frase.huf frase.out
diff -s frase.txt frase.out

./compactador texto.txt texto.huf
./descompactador texto.huf texto.out
diff -s texto.txt texto.out
```

Resultados dos testes

Arquivo	Original (bytes)	Dados Huffman (bytes)	.huf final (bytes)	Redução
frase.txt	100	48	220	dados: 52% / total: cabeçalho domina
texto.txt	4740	2544	2797	41% no arquivo final completo
unico.txt (10 vezes 'A')	10	2	21	edge case (1 símbolo)

Em todos os casos, diff confirmou que o arquivo recuperado pelo descompactador é byte a byte idêntico ao original. O compactador **funciona** para arquivos com qualquer combinação de bytes (o programa lê e grava em modo binário); o descompactador **funciona** desde que o arquivo de entrada tenha sido gerado pelo próprio compactador (a assinatura "HUFF" é verificada).

Facilidades e dificuldades

- A maior dificuldade foi resolver o **caso degenerado de um único símbolo distinto no arquivo (por exemplo, "AAAA...A")**. Sem tratamento especial, a "árvore" seria uma única folha sem código, e o decodificador entraria em laço infinito ao tentar consumir bits. A solução foi forçar a criação de um nó interno com a folha real à esquerda e uma folha fantasma de frequência 0 à direita; assim o símbolo recebe sempre o código 0.
- O **padding de bits no último byte** é uma sutileza clássica de compactadores de bits. Para evitar interpretar os zeros de padding como um símbolo extra, optei por gravar o total de símbolos do arquivo original no cabeçalho e parar a decodificação assim que esse total é atingido.
- A escolha de uma **lista ligada ordenada** em vez de uma heap simplificou bastante o código sem comprometer o desempenho ($n \leq 256$). Como a inserção é estável (em caso de empate a ordem natural dos bytes é preservada), o compactador e o descompactador chegam exatamente à mesma árvore.

- A **separação em módulos** (huffman, bitio, programa principal) deixou o código muito mais fácil de testar. O módulo bitio foi validado isoladamente pela própria roundtrip dos arquivos.

Conclusão

O algoritmo de Huffman foi implementado por completo: contagem de frequências, construção da árvore via fila ordenada, geração da tabela de códigos, escrita do fluxo de bits e leitura inversa para descompactação. Os testes com a frase do enunciado e com o arquivo de texto disponibilizado no EAD demonstram que a compressão é **sem perda e que a tabela exibida na tela contém efetivamente os mesmos** códigos usados na geração do fluxo. O ganho de compressão é coerente com a teoria: depende inversamente da entropia do texto e supera com folga a codificação trivial de 5 bits por caractere descrita no início do enunciado.